

OmniGuard: Two-level Protection Framework for RRAM-based Accelerator with High Efficiency and Flexibility

Zihao Zheng^{*}, Ling Liang^{*}, Yunfan Yang, Jinlong Lin, Meng Li, Zongwei Wang[†], *Member, IEEE* Yimao Cai[†], *Member, IEEE*

Abstract—RRAM-based Deep Neural Network (DNN) accelerators have gained widespread usage in edge devices. However, the security vulnerabilities of RRAM-based accelerators hinder their real application. Current research on safeguarding RRAM-based accelerators predominantly relies on a single-level protection approach. This has resulted in the restriction of its protection scope, the rigidity and lack of generality in the protection method, or has had an impact on the computational efficiency of the system. As a result, it encounters substantial challenges in attaining comprehensive optimization across multiple dimensions, such as universality, the scope of protection, and security-related overheads. In this paper, we develop specific analyses on accelerators and attacks and build graph-based representations. Based on these, we partition the RRAM-based accelerators' security into two levels: on-chip security and off-chip security. Furthermore, we propose a two-level protection framework for RRAM-based accelerators, which is called *OmniGuard*. At the on-chip security level, *OmniGuard* proposes a bit-grained shuffle to achieve protection while using lightweight Benes Networks to maintain the CIM capability. At the off-chip level, *OmniGuard* proposes an RRAM-based AES engine to introduce the AES algorithm into the accelerator with significant acceleration and minimal overhead. Evaluation results demonstrate that *OmniGuard* provides powerful and flexible protection while achieving $1.33\times\sim4.38\times$ speedup and $1.26\times\sim2.65\times$ power savings, with only 5% energy overhead and 3% area overhead.

Index Terms—RRAM-based accelerator, Security Level Partitioning, Bit-grained Shuffle, Benes Network Circuit, RRAM-based AES Engine

I. INTRODUCTION

DEEP Neural Networks (DNNs) consistently achieve superior inference accuracy compared to other machine learning methods, leading to their widespread application in various domains, such as object detection [1], human pose estimation [2], and speech recognition [3]. As technology advances, the computational requirement of DNN models is rapidly increasing. Edge devices encounter significant challenges in achieving fast DNN inference due to limitations

in area and power consumption. Therefore, designing dedicated hardware accelerators for edge scenarios has become a hotspot [4]. Among these specialized accelerators, RRAM-based Computing In Memory (CIM) accelerators are considered the mainstream form due to their non-von Neumann architecture and lower activation voltage and read delay, exhibiting high efficiency and parallelism [5], [6].

RRAM-based accelerators face significant security challenges that limit their practical application [7], [8]. The training of DNNs consumes a lot of time and economic costs. From this perspective, well-trained DNNs can be regarded as a form of intellectual property (IP). Hardware hackers can exploit various methods to attack the accelerator, leading to the distortion and theft of DNN models. This will cause serious economic losses to the model owner.

This insecurity of RRAM-based accelerators comes from two aspects: one is the insecurity of RRAM devices [9], and the other is the insecurity of the architecture [8]. Firstly, due to the non-volatile nature of the RRAM devices, the DNN weights stored on them are retained even when the system is powered off, making them vulnerable to attacks and theft. Secondly, due to limited resources on the accelerator chip, the intermediate results computed at each layer of the DNN model need to be cached, so an off-chip memory needs to be equipped at the architecture level. Under this kind of architecture, the data paths and the off-chip memory are all exposed, which easily becomes a target of attack.

Some researchers have proposed protection schemes for CIM accelerators to solve their security problems. Secure-RRAM [7] proposed a crossbar encryption method based on the XOR algorithm, but its encryption method is relatively simple with limited protection capability. In addition, it also ignores the protection of data paths and off-chip memory. SRA [8] attempts to use special encoding and special computational paradigms (Stochastic Computing) to achieve security protection, but it has resulted in a significant decrease in computational efficiency. RRAM-DES [10] attempts to use mature encryption algorithms (the DES algorithm) for protection but is limited by redundant calculations and the additional time overhead of the encryption algorithm. Chaotic-Weights [11] and ChaoPIM [12] introduce a block-grained shuffle to encrypt crossbars. Similarly, this method needs to shuffle the computation kernel in the training process, which also needs a special training method. In addition, this kind of method also ignores the protection of the intermediate results.

Considering the diversity of security vulnerabilities and protection methods in accelerators, it is necessary to divide

^{*} means equal contribution and [†] means the corresponding author.

This work is supported by National Key R&D Program of China: 2024YFB4505800, National Natural Science Foundation of China (62341407, 62322401, U24B20166, 62406008), in part by the "111" Project (B18001). This work mainly finished at School of Integrated Circuits, Beijing Advanced Innovation Center for Integrated Circuits, Peking University, Beijing, China. Zihao Zheng is with the School of Computer Science, Peking University, Beijing, China. Ling Liang, Yunfan Yang, Meng Li, Zongwei Wang <wangzongwei@pku.edu.cn> and Yimao Cai <caiyimao@pku.edu.cn> is with the School of Integrated Circuits, Beijing Advanced Innovation Center for Integrated Circuits, Peking University, Beijing, China. Jinlong Lin is with the School of Software and Microelectronics, Peking University, Beijing, China. Zongwei Wang is also with the YanXin MicroElectronics Co., Ltd. (YXME), Shanghai, China.

TABLE I
COMPARATION BETWEEN *OmniGuard* AND OTHER RELATED WORKS.

Works	Security level	Training-Free	Efficiency	Robust	Universality	On-Chip Security	Off-Chip Security
Secure-RRAM [7]	Single Level	✓	✓	✗	✓	✓	✗
SRA [8]	Single Level	✗	✗	✓	✓	✓	✓
RRAM-DES [10]	Single Level	✓	✗	✓	✓	✗	✓
Chaotic-Weights [11]	Single Level	✓	✓	✓	✗	✓	✗
ChaoPIM [12]	Single Level	✓	✓	✓	✗	✓	✗
<i>OmniGuard</i>	Two level	✓	✓	✓	✓	✓	✓

security levels and adopt the most appropriate security protection strategies at different levels before protecting RRAM-based accelerators. The current protections for RRAM-based accelerators are predominantly single-level and adopt a uniform approach, lacking differentiated protection measures for various security vulnerabilities and attacks. This not only causes the scope of protection to be restricted but also limits the flexibility of the system. In response to this situation, this article attempts to classify security levels and develop the most suitable protection strategies for different levels.

However, achieving such multi-level security protection faces challenges. Firstly, there are differences between various attack methods. The implementation of some attack methods requires different attack targets and even different time points of occurrence. In this situation, how to divide security levels has become a challenge. Secondly, due to the diversity of protection methods, choosing the most suitable protection strategy for each security level has also become a challenge.

To address these issues, this article first provides a detailed analysis of accelerators and attack methods. We develop a data-flow-based analysis for accelerators and identify their security vulnerabilities. Moreover, we perform a target-based analysis for attacks. Based on these two analyses, we build a unified graph-based representation and segment the security of RRAM-based accelerators into two levels: off-chip security and on-chip security. After the determination of security level partitioning, we propose a flexible and efficient security framework called *OmniGuard*. *OmniGuard* adopts a multi-level security design for RRAM-based accelerators. At the on-chip level, *OmniGuard* proposes a bit-grained shuffle for crossbars encryption while maintaining CIM capability with the help of Benes Networks. At the off-chip level, *OmniGuard* develops an RRAM-based AES engine to achieve fast, efficient, and powerful encryption, which provides robust protection for the data paths and off-chip memory. In summary, our contributions are three-fold:

- We put forward the concept of partitioning security levels before devising protection mechanisms for RRAM-based accelerators. Furthermore, we develop a distinctive analytical method to deconstruct the intricacies of security vulnerabilities and attacks. This method enables us to generate a unified graphical representation upon which the partitioning of security levels is accomplished.
- We propose a multi-level security framework called *OmniGuard*. At the on-chip level, *OmniGuard* implements a bit-grained shuffle that provides sufficient protection while

maintaining CIM capability. At the off-chip level, *OmniGuard* implements a novel RRAM-based AES engine that enables fast and efficient encryption. We believe *OmniGuard* will create favorable conditions for the future development of RRAM-based accelerators.

- We provide experiments to evaluate *OmniGuard*. As experiment results show, *OmniGuard* achieves $1.33 \times \sim 4.38 \times$ speedup and $1.26 \times \sim 2.65 \times$ power savings while providing effective protection, compared to SOTA security designs, with only 5% power overhead and 3% area overhead.

II. BACKGROUNDS AND MOTIVATIONS

A. RRAM-based CIM Accelerator

1) **RRAM-based CIM Principle:** RRAM crossbars enable efficient in-situ Vector Matrix Multiplication (VMM), qualifying them as strong candidates for CIM implementations [13], [14]. In RRAM-based CIM systems, DNN weights are encoded into the conductivity of RRAM cells, and input activations are encoded as voltages. Per Ohm's law and Kirchhoff's law, the currents sensed from the RRAM crossbar directly represent VMM results. Adopting RRAM-based CIM significantly lowers DNN computational complexity, minimizes data movement, and speeds up inference [15], [16].

2) **RRAM-based Accelerator's Architecture:** The common architecture of RRAM-based accelerators is presented in Fig. 1. A single chip integrates multiple Tiles, each consisting of multiple Processing Elements (PEs), an Shift&Add (S+A) module, a ReLU Unit, a pooling module, and a controller. Each PE contains multiple crossbars, a Sample and Hold (S&H) module, an S+A module, input and output registers, and several Analog-to-Digital Converters (ADCs).

3) **Necessity of Off-Chip Memory:** Due to the limited on-chip storage resources of accelerators, the intermediate results generated by DNN models during inference must be transferred out and cached in off-chip memory. Consequently, RRAM-based CIM accelerators frequently rely on off-chip memory modules for such caching operations [5], [6]. During the DNN inference process, continuous data transmission of intermediate results thus occurs between the accelerator and the off-chip memory.

B. Various Vulnerabilities of RRAM-based Accelerators

1) **Vulnerability Caused by RRAM Cells:** As Fig. 1 shows, an RRAM cell features a three-layer structure, with a resistive switching layer situated between the top and bottom electrodes [9], [17]. Each cell's resistance/conductivity

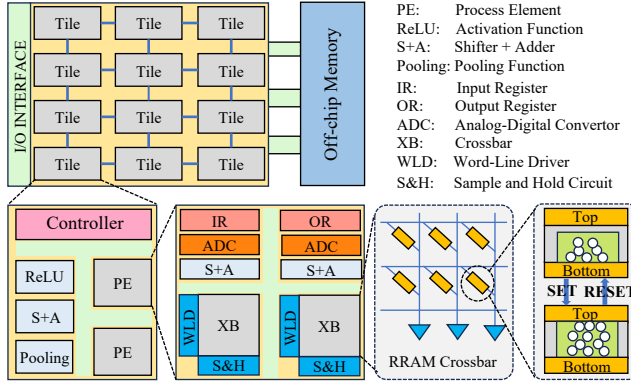


Fig. 1. The Hardware Architecture of RRAM-based Accelerators.

is determined by the oxygen vacancy filament that connects the two electrodes, and this resistance/conductivity is used to represent binary information. This information preservation method based on oxygen vacancy is non-volatile, and the information will not be lost even if the system is powered off. However, when RRAM cells are programmed to act as DNN weights, this persistence renders them vulnerable to theft [12].

2) **Vulnerability Caused by the Off-Chip Memory:** The off-chip memory in RRAM-based accelerator also introduces a security vulnerability [8]. Data paths connecting the accelerator to off-chip memory are left exposed, enabling straightforward interception of plaintext intermediate results [18], [19]. As previous studies show [8], [20], if the intermediate results are intercepted, attackers can leverage them to reversely infer the weights, resulting in model leakage. For example, assume W_l is the weights of layer l . A_{l+1} and A_l are the intermediate results of layer l and layer $l + 1$, respectively. If attackers can obtain A_{l+1} and A_l , then the weights can be acquired through $W_l = A_{l+1} \cdot A_l^{-1}$. Additionally, beyond the data paths, the off-chip memory itself is also vulnerable to exposure. Specifically, certain attack methods can directly target the memory to extract stored information [21], [22].

C. Diversity Attacks on RRAM-based Accelerators

1) **Attacks' Difference in Multiple Aspects:** RRAM-based accelerators exhibit various vulnerabilities, leading to multiple types of attacks. These attack methods vary in several aspects, including implementation methods, attack time points, and attack targets. First, the implementation methods differ significantly; some attacks [8] necessitate direct physical contact with the hardware, while others can be executed remotely with the aid of additional devices or tools [23]–[25]. Second, the attacks are executed at different time points. Some attacks arise while the system is running [26], [27], while others are conducted offline. Third, the targets of these attacks vary widely. Some methods specifically attack the RRAM crossbars, while others target off-chip memory.

2) **Unclear Safety Boundaries Caused by Diverse Attacks:** These diverse attacks on RRAM-based accelerators create challenges in defining a clear safety boundary. Existing studies typically focus on defending against a single type of attack, yet such a targeted design tends to be ineffective when confronting other attack vectors. This limitation encourages us to adopt a

layered approach: we re-analyze the characteristics of various attack methods and then implement comprehensive protection measures across different levels.

D. Existing Signle-Level Protection Scheme

1) **Encryption-based Protection:** Some studies integrate encryption methods into the protection of RRAM-based accelerators. Secure-RRAM [7] employs XOR-based encryption to protect RRAM crossbars, yet this method is relatively simple and fails to cover the protection of off-chip memory and data paths. RRAM-DES [10] applies the mature DES algorithm for RRAM protection, but mature encryption algorithms typically lead to high computational costs and latency, which may impair accelerator performance.

2) **Paradigm-based Protection:** Some studies apply novel computing paradigms to RRAM-based accelerators for secure protection. For instance, SRA [8] adopts the Stochastic Computation paradigm in such accelerators and shows good protection effects. However, this approach fails to safeguard off-chip memory and data paths. In addition, the high computational cost of the stochastic computation paradigm reduces the accelerator's overall efficiency.

3) **Shuffling-based Protection:** Some work adopts shuffling to disrupt the calculation order in accelerators and uses this order as a secure key. For instance, ChaoPIM [11] and Chaotic-Weights [11] randomly change the order of convolution kernels during training to strengthen protection. Since convolution kernels correspond to weight blocks in RRAM crossbars, this method can be seen as block-grained shuffling. However, this coarse-grained shuffling struggles to safeguard the distribution of weights, which are regarded as a part of sensitive information, and also fails to address the protection of data paths and off-chip memory. Moreover, it depends on specialized DNN training methods to realize block-grained shuffling, which reduces its flexibility and universality in practical scenarios.

4) **Shortage of Single-Level Protection:** It is evident that combining the diverse attacks with the various vulnerabilities of the accelerators makes the problem extremely complex. Existing single-level encryption solutions cannot address this complexity or achieve optimal performance across all dimensions. To address these, we propose a two-level protection framework called *OmniGuard*. We conduct a comparative analysis of *OmniGuard* and existing methods across multiple aspects, with results summarized in Tab I. The findings show *OmniGuard* has advantages in all key aspects, highlighting the superiority of two-level protection approaches.

E. Related Tools and Methods in OmniGuard

OmniGuard adopts a two-level protection method, which involves some advanced technologies. We briefly introduce these technologies.

1) **Benes Networks:** In *OmniGuard*, we adopt a specialized circuit known as the Benes Network (BN) [28], [29]. Composed of multiple multiplexers (MUXs), the BN enables arbitrary signal shuffling. This structural simplicity, together with its advantages of low power consumption and high speed, has facilitated its widespread application in communication

networks and digital signal processing. For example, an 8-bit BN requires only 48 MUXs, which are divided into six groups; each group is regulated by a distinct control vector to realize arbitrary signal reordering. Taking the input signal sequence "ABCDEFGH" as an instance: under the control of the vector sequence "1101", "0010", "0110", "0010", "0100", and "0010", the BN can rearrange the output into the sequence "CFGHBDEA". We used BN in *OmniGuard*, which will be detailed in Section IV-A3.

2) **The AES Algorithm:** In *OmniGuard*, we also utilize the AES algorithm [30], [31]. The AES algorithm consists of four steps. **Step ① (RoundKey Generation):** This step takes a 128-bit key as input and generates eleven 128-bit RoundKeys as output. It needs a constant ten-length list called RCon, and each element in RCon is 32 bits. **Step ② (Initial Round):** In this step, the AES algorithm achieves an operation named **AddRoundKey**, which uses 128-bit plaintext to perform an XOR operation with the corresponding RoundKeys and then divides into 16 States. Each State is 8 bits. **Step ③ (Main Rounds):** This step consists of ten rounds, and each round contains four operations:

- **Operation ① (SubByte).** In this operation, the input is 16 States, and the output is 16 replaced States. Each State is replaced by the corresponding number in S_BOX. S_BOX is a constant 16×16 matrix, and each element in it is 8-bit.
- **Operation ② ShiftRow.** 16 replaced State is arranged as a 4×4 matrix. Then this operation cyclically shifts the bytes of each row in the State matrix to the left. Among them, the first row of the matrix remains unchanged, the second row is shifted left by 1 byte, the third row is shifted left by 2 bytes, and the fourth row is shifted left by 3 bytes.
- **Operation ③ MixColumn.** This operation performs a linear transformation on the bytes in each column to increase diffusion. Specifically, the 4×4 matrix is multiplied by MIX_C matrix. MIX_C is a constant 4×4 matrix, with each element requiring 8 bits. Each element multiplication involves polynomial multiplication.
- **Operation ④ AddRoundKey.** The details of this operation are as described earlier.

Step ④ (Final Round): This step is similar to Step ③ but omits the MixColumn operation, so it only includes three operations: SubByte, ShiftRow, and AddRoundKey. After these operations, the encryption process of AES algorithm is finished, and the output State Matrix is the ciphertext. We introduce AES algorithm in RRAM-based accelerators and design a special AES engine, which will be detailed in Section IV-B.

III. SPECIFIC ANALYSIS FOR SECURITY-LEVEL PARTITIONING

As aforementioned, single-level protection is insufficient and often fails to achieve optimal performance. An effective solution is to divide security levels and apply distinct protection methods to each. However, defining the boundaries of these levels is a major challenge. To tackle this, we conduct specific analyses and construct a unified graph-based representation for security level partitioning.

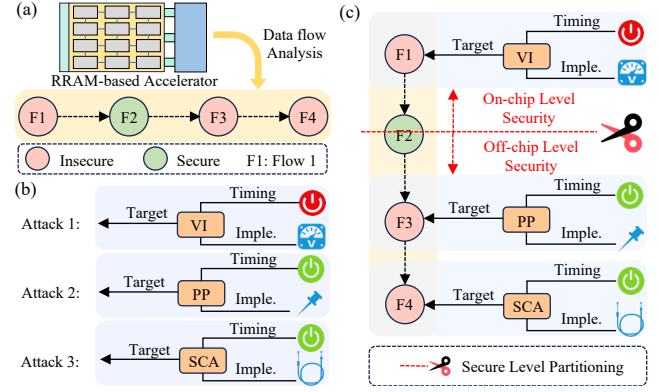


Fig. 2. (a) Dataflow-based Analysis for accelerators. (b) Target-oriented Analysis for Attacks. (c) Secure Level Partitioning.

A. Dataflow-based Analysis for Accelerators

To effectively partition security levels, analyzing the security vulnerabilities of accelerators is essential. These vulnerabilities, however, are inherent hardware characteristics, making them difficult to quantify and model. We propose a dataflow-based analysis for accelerators, dividing the overall dataflow in RRAM-based accelerators into four parts.

1) Dataflow Dividing:

a) **Flow 1 (F1):** Assume an RRAM crossbar are denoted as $\mathcal{R}_{(i,j)}$, while $i, j = 1, 2, \dots, n$. And the input activations are compiled to voltage $\mathcal{V}_i$. The output current $\mathcal{I}_j$ represents the computing results. We use $Crossbar(\cdot, \cdot)$ to represent the computation process in the crossbar. $\xleftarrow{\text{on-on}}$ means this dataflow starts from an on-chip component (RRAM Crossbars in PEs) and ends at an on-chip component (PEs in each Tile). F1 can be written as Eq. 1.

$$\mathbf{F1:} \quad \{\mathcal{I}_j\}_{j=1}^n \xleftarrow{\text{on-on}} \{Crossbar(\mathcal{V}_i, \mathcal{R}_{(i,j)})\}_{i,j=1}^n, \quad (1)$$

$$\text{s.t.} \quad Crossbar(\mathcal{V}_i, \mathcal{R}_{(i,j)}) = \sum_{i=1}^n \mathcal{V}_i * \mathcal{R}_{(i,j)}.$$

b) **Flow 2 (F2):** Then, the sensed current $\mathcal{I}_j$ is sent out of PEs and achieves some specific computation function, such as pooling $Pool(\cdot)$, ReLU $ReLU(\cdot)$ and Shift and Add $Shift\&Add(\cdot)$. We use function $\mathcal{F}(\cdot)$ to unified represent these functions. Note that these functions' circuit located in tiles, which are also on-chip components. We use $\mathcal{M}_k$ to represent the intermediate results after these specific functions, while $k = 1, 2, \dots, t$. ϑ means the cycle times of function $\mathcal{F}(\cdot)$, because each $\mathcal{I}_j$ may experience multiple functions, e.g., $\mathcal{F}^2(\mathcal{I}_j) = Pool(ReLU(\mathcal{I}_j))$. F2 can be written as Eq. 2.

$$\mathbf{F2:} \quad \{\mathcal{M}_k\}_{k=1}^t \xleftarrow{\text{on-on}} \{\mathcal{F}^{\vartheta}(\mathcal{I}_j)\}_{j=1}^n, \quad (2)$$

$$\text{s.t.} \quad \mathcal{F}(\cdot) = ReLU(\cdot) \text{ or } Pool(\cdot) \text{ or } Shift\&Add(\cdot).$$

c) **Flow 3 (F3):** Then, the intermediate results $\{\mathcal{M}_k\}_{k=1}^t$ are sent out of the accelerator through the data paths. We use $\mathcal{D}_{path}^k$ to represent the transmission process on the data paths. F3 can be written as Eq.3. $\xleftarrow{\text{off-on}}$ reflects that this process occurs on these data paths.

$$\mathbf{F3:} \quad \{\mathcal{D}_{path}^k\}_{k=1}^t \xleftarrow{\text{off-on}} \{\mathcal{M}_k\}_{k=1}^t. \quad (3)$$

d) **Flow 4 (F4)**: After that, the intermediate results are cached in the off-chip memory for future reuse. If we use $\{Mem_{(q,v)}\}_{q=1,v=1}^s$ to represent the off-chip memory and s means the memory size, F4 can be written as Eq. 4. Note that this process starts from the data paths and ends at the off-chip memory, which is represented as $\xleftarrow{\text{off-chip}}$.

$$\mathbf{F4}: \{Mem_{(q,v)}\}_{q=1,v=1}^s \xleftarrow{\text{off-chip}} \{\mathcal{D}_{path}^k\}_{k=1}^t. \quad (4)$$

2) **Graph-based Representation for Dataflows.**: The accelerator repeats these dataflows to complete each layer's calculations, and analyzing these dataflows enables easy identification of security vulnerabilities. Despite occurring in hardware encapsulation, F1 remains insecure due to RRAM's non-volatile nature. F2 is safe; it operates within hardware encapsulation, and data loss when the system is powered off ensures security. In contrast, F3 is unsafe because its associated data path is exposed, and F4 is also insecure as there are attack methods targeting off-chip memory. Additionally, to facilitate future secure level partitioning, we develop a unified representation of these dataflows by establishing a graph-based model (shown as Fig 2 (a)). In this graph, dashed lines represent the sequential process and different colors indicate whether the dataflow is safe, which will support subsequent secure level partitioning.

B. Target-Oriented Analysis for Attacks

There are various attack methods for RRAM-based accelerators, and their implementation methods, targets, and timing of these attacks are all different. Therefore, we first summarize the following three most common attacks.

1) Summarization for Various Attacks:

a) **Voltage Injection (VI)**: The attack target of VI is the RRAM crossbar, which can be exploited even when the system is powered off [7], [11], [12]. This attack necessitates physical contact with the hardware [26]. By injecting voltage pulses into the accelerator and monitoring the output current, an attacker can ascertain the resistance of the RRAM, resulting in the leakage of DNN weights.

b) **Physical Probe (PP)**: The attack target of PP is the data path between the accelerator and off-chip memory. This attack takes place during system operation and requires physical contact for execution [8]. By placing micro but delicate probes on the data paths connecting the accelerator and off-chip memory, attackers can intercept intermediate results transmitted along these routes.

c) **Side Channel Attack (SCA)**: The primary target of SCA is off-chip memory, which can be executed during system operation without requiring physical contact [21]. By monitoring and analyzing the power consumption or electromagnetic radiation emitted by the off-chip memory, attackers can infer the intermediate results of the DNN model. This attack method typically necessitates specialized equipment (*i.e.*, electromagnetic coil) for effective implementation [23].

d) **Other Attacks**: Considering the cost and feasibility of various attacks, we will not address other potential threats in this paper. For instance, while SCA could theoretically target RRAM-based accelerators directly instead of off-chip memory, the complexity of the accelerator architecture demands substantial resources for an effective attack. In practice, despite

the availability of these potential attack methods, attackers generally do not choose to utilize them. Consequently, we focus on the most prevalent attacks and overlook those that involve high costs or significant complexity.

2) **Graph-based Representation for Attacks**: Furthermore, to standardize the representation of security vulnerabilities related to accelerators, we explore the use of graph-based modeling for such attack scenarios. Specifically, each attack method is modeled as a node in the graph, while the distinct attributes of each attack method are represented as edges. Given the requirement to integrate with the graph presented in Section III-A, we adopt the attack target as the core orientation. This design choice enables the attack target to serve as a potential adhesive joint for the future integration of the two graphs.

As shown in Fig. 2 (b), nodes in the graph correspond to specific attack methods, while the edges represent the attack target, timing, and implementation method. 'Timing' means the time when the attack occurred (*e.g.*, VI attacks occurred on powered-off.). 'Implementation method' means the implementation and deployment methods of attacks (*e.g.*, SCA attacks depend on specific devices but do not need physical contact.). 'Timing' is represented by a button in Fig. 2 (b). The green button indicates that the attack occurs when the system is running. The red button indicates that the attack occurs when the system is powered off. 'Implementation method' is represented by icons, including voltage generator (used in VI), probe (used in PP), and electromagnetic coil (used in SCA).

The edge of the attack target is arranged separately on one side, waiting for future adhere. Under this unified graph-based representation, we have the opportunity to combine accelerator analysis with attack analysis and explore the optimal segmentation of the graph to achieve security-level partitioning in Section III-C.

C. Security-Level Partitioning

1) **Unified Representation based on Two Graphs**: Based on the graphs established in Section III-A and Section III-B, we can glue the two graphs together to obtain an overall expression that includes both accelerator security vulnerabilities and attack methods. Thanks to our previous representation approach, we find that nodes in the data flow graph and edges representing the target in the attack graph can be glued together to achieve graph adherence.

2) **Security-Level Segmentation based on the Unified Representation**: Fig. 2 (c) illustrates the results of adhering these two graphs. From Fig. 2 (c), we observe the emergence of a 'vacuum zone' around F2, which serves as a natural partitioning belt. We partition the entire glued graph at this point, with the upper half encompassing F1 and the lower half comprising F3 and F4. Based on our analysis of attacks, we can use the segmentation points in this graph to represent the partitioning of security levels. The attack targets associated with the methods in F1 focus on on-chip modules, allowing us to categorize this segment as indicative of on-chip security. In contrast, the targets of the attack methods in F3 and F4 pertain

to off-chip components, making this segment representative of off-chip security.

Through this approach, we achieve a natural partitioning of security levels for RRAM-based accelerators. Instead of imposing artificial cuts, we identify natural segmentation bands within the RRAM architecture using a unified graph representation. This method enhances the reasonableness and interpretability of our partitioning. Additionally, the security protections implemented based on this partitioning are more likely to achieve optimal performance.

IV. TWO-LEVEL PROTECTION FRAMEWORK

Based on the security-level partitioning, we develop a two-level protection framework called *OmniGuard* for RRAM-based accelerators. *OmniGuard* achieves protection on both two levels (on-chip and off-chip) respectively and integrate system optimization technologies.

A. On-Chip Level Protection

1) **Requirement of finer-grained Shuffling:** According to our security level partitioning, we should first consider on-chip security and select appropriate protection policies. The main object of on-chip security is the RRAM crossbar. Some works have proposed encryption schemes for RRAM storage, for example, utilizing block-grained shuffling to achieve protection [11].

However, block-grained shuffling does not encrypt each weight, but simply swaps the order of the convolutional kernel (corresponding the layout of the weight blocks in the RRAM crossbar). This means that attackers can steal the overall weight distribution. Further, attackers can gradually restore the correct layout based on the weight distribution through a limited number of attempts, or retrain a model with the same weight distribution, resulting in model leakage. Additionally, the block-grained shuffling [11] requires special training algorithms to reorder kernels, leading to additional overhead and further limiting its generality. Therefore, a finer-grained and training-free shuffling method is required for RRAM crossbar protection.

2) **Bit-Grained Shuffling for On-chip Protection:** After careful consideration, we propose a bit-grained shuffling method that balances complexity and universality without requiring special training methods or incurring additional training costs. The bit-grained shuffling method consists of two steps: row shuffling and column shuffling. Fig. 3 (a) illustrates the details of the bit-grained shuffling.

a) **Row shuffling:** Specifically, taking a 4×4 sub-array as an example, DNN weights are typically stored in the format illustrated by Pattern ①. During the offline encryption phase, we first perform random row shuffling on the RRAM crossbar. After this row shuffling operation, the weight distribution in the RRAM crossbar is transformed from Pattern ① to Pattern ②. By comparing Pattern ① and Pattern ②, it can be observed that naive row shuffling only alters the order of weights, without implementing encryption for individual weight values themselves. At this point, we need to adjust the order of the input signals from 'ABCD' to 'CDBA' to ensure the correct CIM output.

b) **Column shuffling:** To address the aforementioned limitation, we further introduce random column shuffling on the RRAM crossbar. As a result, the weight distribution is modified again, this time from Pattern ② to Pattern ③. It is obvious that after row shuffling and column shuffling, both the weight values and their original order are well protected. Compared with existing methods, the proposed bit-grained shuffling scheme achieves a higher level of security protection. All calculations related to the shuffling are performed offline. We directly program the RRAM crossbar according to the Pattern shown in ③, which avoids the need for online order swaps and thus eliminates additional overhead. However, given that the output signal is a posteriori (*i.e.*, dependent on computational processes), the computation result cannot be recovered in the same manner as exchange input signal order. Consequently, following column shuffling, the capability of the CIM architecture is impaired, ultimately leading to erroneous calculation outcomes.

3) **Recover the CIM Capability by Benes Network:** Using a specific algorithm to recover Compute-In-Memory (CIM) capability introduces significant extra computation, compromising the RRAM accelerator's high energy efficiency. Thus, we adopt a direct circuit-level strategy for CIM restoration. Considering RRAM's low power consumption and high read-write speed, the circuit needs to achieve signal rearrangement while maintaining high bandwidth and low power consumption. Therefore, we propose a method that integrating Benes Network (BN) [28], [29] into RRAM crossbar to efficiently recover the CIM capability.

a) **Integrating BN into RRAM Crossbar:** As shown in Fig. 3 (b), we add BNs on RRAM crossbars' input/output side to achieve permutation of the input and output signal positions, ensuring the correct CIM result is recovered. We refer to the input side BN as IBN (Input BN) and the output side BN as OBN (Output BN). IBN and OBN require correct control vectors to restore CIM results. The control vectors are treated as encryption keys for crossbar encryption. Based on the shuffling order, we pre-compute the corresponding control vectors. Specifically, assume the row shuffling and shuffling as $\mathbb{O}_{\text{rol}}(W)$ and $\mathbb{O}_{\text{col}}(W)$, we can use bit-wise greedy searching algorithm to compute the corresponding control vectors: $CV \leftarrow \text{Search}(\mathbb{O}_{\text{col}}(\mathbb{O}_{\text{rol}}(W)))$. Note that this process is done offline, and its time overhead is not introduced into the DNN inference. And the corresponding control vectors are saved on chip. We use volatile memory to build secure control vector buffers on chip. Since these volatile memories are integrated within the chip encapsulation and exhibit volatile properties, the control vectors' storage is regarded as secure.

b) **BN Details and Overhead Discussion:** We discuss the details and overheads of BN with different precision, as shown in Tab. II. We implemented different precision BN circuits based on Verilog and evaluated their overhead using DC. We determined the number of IBNs and OBNs based on the common RRAM Crossbar size, and estimated the size of the control vector. It can be seen that the overhead of BN is minimal at low precision (INT4). As the precision increases (FP16), the overhead of BN rises rapidly. Considering the overhead control of BN and the quantization precision commonly

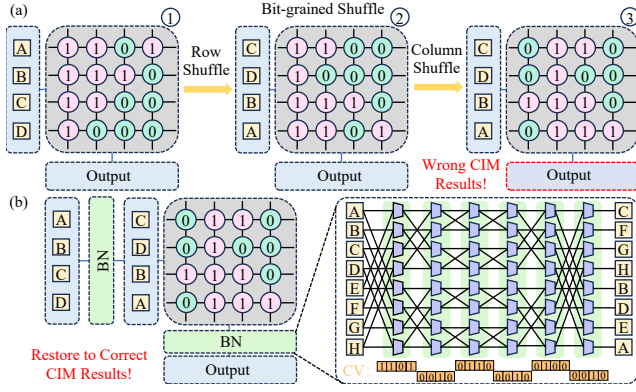


Fig. 3. (a) An Overview of the Bit-grained Shuffling. (b) Using Benes Networks to Restoring the CIM capability.

TABLE II
DETAILS AND OVERHEAD OF BN UNDER DIFFERENT PRECISION

W-Bit	BN Details			CV	Total Overhead	
	BN-Bit	MUX	Num		Power	Area(mm ²)
INT4	4bit	16	64	6B	0.05mW	1.76×10^{-4}
INT8	8bit	12	32	12B	0.16mW	5.89×10^{-4}
FP16	16bit	160	16	20B	0.78mW	2.88×10^{-3}

used by accelerators, in this paper, we choose to use 8-bit BN in *OmniGuard*. For lower quantization accuracy, we can reuse the 8-bit BN for flexible processing without modifying the architecture or circuits. Specifically, to achieve arbitrary bit-grained shuffling of INT4 weights, 8 Bit BN can shuffle two INT4 weights in a single operation. Therefore, our choice of BN can be flexibly compatible with lower weight quantization precision.

c) **BN's Control Vector Reusing:** If each BN uses a different control vector, although the security is guaranteed, it will cause a large on-chip buffer and memory overhead. We propose a control vector reuse method to make an optimal trade-off between safety and efficiency. Assuming weights are quantized to INT8 and 8-bit BNs are employed, *OmniGuard* designates every 8 rows and 8 columns as an encryption sub-array. We make different sub-arrays reuse the same control vectors. Specifically, if the number of IBNs is n and the number of OBNs is n , n^2 sub-arrays use the same shuffling order and the same control vectors. After our extensive experimental testing, we can achieve an optimal balance between security and overhead when $n = 8$. This design not only reduces the number of control vectors but also decreases its memory requirements. In addition, this approach does not affect security. Without a control vector, an attacker needs to make 2^{24} times of attempts to crack each sub-array.

This section presents a robust on-chip security mechanism that uses bit-grained shuffling and restores CIM functionality via BN circuits. We further detail associated design aspects (e.g., overhead analysis and control vectors). This approach balances protection scope, implementation complexity, and universality, and dispenses with specialized training.

B. Off-Chip Level Protection

Off-chip security issues include vulnerabilities in data paths and insecurity of off-chip memory. Addressing these two

TABLE III
LATENCY RATION OF OPERATIONS IN THE AES ALGORITHM.

	Steps	Operations	Exe. Counts	Lat. Ratio
AES	Step ①	RoundKeyGen	1	9.30%
	Step ②③④	AddRoundKey	11	9.15%
		SubBytes	11	4.76%
		ShiftRow	11	0.82%
	Step ②③	MixColumn	10	75.97%

vulnerabilities separately would introduce significant complexity. Additionally, off-chip storage is usually user-determined, which makes it difficult for accelerator providers to require users to actively encrypt their off-chip storage. Thus, an effective solution is to adopt mature encryption algorithms to defend against both PP attacks and SCA attacks.

1) Chance and Challenge of Integrating AES Algorithm:

We choose to introduce the online AES algorithm in the RRAM accelerator. The AES algorithm has high efficiency and security, and can be applied in a variety of scenarios with good flexibility, making it our first choice. However, mature encryption algorithms require iterative computations [32], [33]. For instance, Fig. 4 (a) illustrates the overall process of the AES algorithm. It involves ten iterations, each with matrix-level computational complexity. Consequently, directly implementing mature encryption algorithms in RRAM accelerators is not feasible, as it would significantly slow down the accelerator's performance. To address this, we propose a novel RRAM-based AES engine to accelerate AES calculations.

2) **AES Algorithm's Bottleneck and Characteristic:** The computational bottleneck of the AES algorithm is not clear, which complicates the design of its acceleration engine. Therefore, we conduct actual testing and analysis of the AES algorithm first. We measure the latency of each operations in the AES algorithm on a consumer-grade CPU (AMD Ryzen 7 6800H), as shown in Tab. III. Specifically, the RoundKeyGen operation runs once, contributing only 9.30% of total latency. Though the AddRoundKey, SubBytes, and ShiftRow operations execute 11 times, their latency is low due to inherent operational simplicity. The MixColumn operation, which involves complex matrix calculations, accounts for 75.97% of the delay and is the AES algorithm's main bottleneck.

Moreover, we find that the AES algorithm requires storing a large number of constants during the encryption and decryption process, including RCon, S_BOX, MIX_C, I_S_BOX and I_MIX_C. As presented in Tab. IV, RCon is a list of length 10 and serves a function in the preparation phase. Each number in this list is 32 bits. S_BOX and I_S_BOX

TABLE IV
CONSTANTS IN THE AES ALGORITHM.

Phase	Constant	Type	Num.	Element Size	Total
Prepare	RCon	List	10	32 bit	320 bit
Encrypt	S_BOX	Matrix	16×16	8 bit	2048 bit
	MIX_C	Matrix	4×4	8 bit	128 bit
Decrypt	I_S_BOX	Matrix	16×16	8 bit	2048 bit
	I_MIX_C	Matrix	4×4	8 bit	128 bit

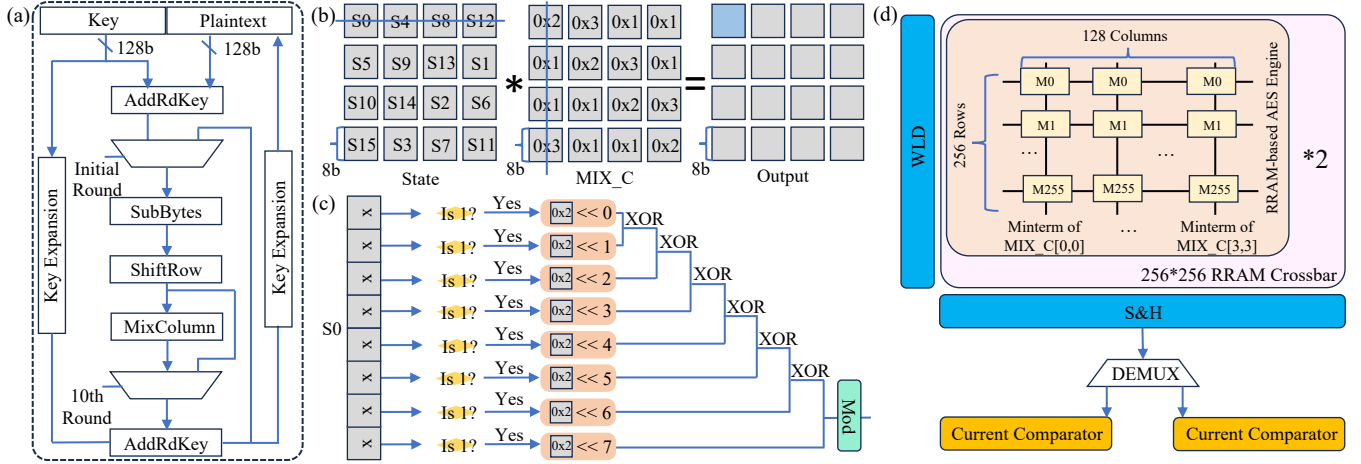


Fig. 4. (a) Details of the AES Algorithm. (b) Details of the MixColumn Step. (c) Calculation Disassembles of the MixColumn Step. (d) The Architecture of the RRAM-based AES Engine.

are 16×16 matrices, where each element is 8 bits. MIX_C and I_MIX_C are 4×4 matrices, and each element within these matrices is also 8 bits. These constants require multiple times of loading/offloading during the AES encryption/decryption process, resulting in a large amount of data movement, which affects power consumption and speed.

We conclude two key aspect of achieve AES algorithm acceleration: (1) Optimizing the Mixcolumn operation in AES (2) Reducing the data movements of those constants in AES. We propose an RRAM-based AES engine, optimizing both aspects simultaneously.

3) **RRAM-based Storage of AES Constants:** Considering the requirements for device consistency and storage constants, we chose RRAM as the base to design the AES acceleration engine due to its favorable characteristics, such as low operating voltage and read-out latency. In addition, designing the AES engine based on RRAM can also maintain the consistency of the non-volatile device with the accelerator, which brings convenience to the actual tape-out. From Tab. IV, the constants in the AES algorithm require a total of 4,672 bits of storage space. For common 256×256 RRAM crossbars, a single crossbar is sufficient to accommodate these constants. This RRAM-based storage scheme not only ensures the consistency of the devices throughout the entire accelerator but also significantly reduces data movements.

4) **RRAM-CIM-based AES Engine:** Prior studies about AES engines based on non-volatile memory (e.g., IM-CRYPTO [34], EIM-AES [35], AIM [36]) all implemented the AES algorithm via the Logical-In-Memory paradigm, which suffers from low flexibility, demands large logical trees adjacent to non-volatile memory, and exhibits low computational efficiency. In contrast, the AES engine proposed in this study computes the AES MixColumn operations based on CIM architecture. Specifically, we decompose MixColumn's complex polynomial multiplication to simple format, which enables leveraging the analog summation capability of RRAM crossbars for acceleration. This design not only achieves higher efficiency but also eliminates the need for additional logic modules within the memory.

a) **Decomposition of MixColumn Operations:** We chose to perform a computational decomposition of MixColumn to utilize CIM paradigm to achieve acceleration. We use a cyclic shift-based approach to implement the polynomial multiplication of MixColumn, the details of which are shown in Fig. 4 (b). The State matrix multiplies the MIX_C matrix, and elements make polynomial multiplication. For example, Eq. 5 shows how to achieve polynomial multiplication between S_0 and 0x2. S_0 means the first number in the State matrix. 0x2 is the first number in the MIX_C matrix.

$$\begin{aligned} R_0^{\{0,0\}} &= S_0[0] << 0 \quad \text{if } 0x2[0] == 1, \\ &\vdots \\ R_7^{\{0,0\}} &= S_0[7] << 7 \quad \text{if } 0x2[7] == 1. \end{aligned} \quad (5)$$

Fig. 4 (c) illustrates the details of the computation disassembly of the MixColumn operation. We use $S_0[i]$ to represent the i -th bit of S_0 which is an 8-bit number. $R_0^{\{0,0\}}, \dots, R_7^{\{0,0\}}$ stands for the shift results of 0x2. And the $Out^{\{0,0\}}$ represents the element located at {0,0} in the output matrix as Eq. 6 shown. mod is a simple decision-shift function.

$$Out^{\{0,0\}} = mod(R_0^{\{0,0\}} \oplus R_1^{\{0,0\}} \oplus \dots \oplus R_7^{\{0,0\}}). \quad (6)$$

According to Eq. 7, the XOR operation can be decomposed into AND and OR operations. We further decompose XOR operations from $R_0^{\{0,0\}}$ to $R_7^{\{0,0\}}$, like Eq. 8. Then it becomes OR operations from $m_0^{\{0,0\}}$ to $m_{255}^{\{0,0\}}$. $m_0^{\{0,0\}}$ to $m_{255}^{\{0,0\}}$ means the 256 minterm of the 8-bit result.

$$R_0^{\{0,0\}} \oplus R_1^{\{0,0\}} = R_0^{\{0,0\}} \overline{R_1^{\{0,0\}}} + \overline{R_0^{\{0,0\}}} R_1^{\{0,0\}}. \quad (7)$$

$$R_0^{\{0,0\}} \oplus \dots \oplus R_7^{\{0,0\}} = m_0^{\{0,0\}} + \dots + m_{255}^{\{0,0\}}. \quad (8)$$

b) **AES Engine Design:** Fig. 4 (d) illustrates the architecture of the proposed RRAM-based AES engine. Unlike existing AES engines' online XOR operation computing, we pre-compute $m_0^{\{0,0\}}, m_1^{\{0,0\}}, \dots, m_{255}^{\{0,0\}}$ and store them in the RRAM crossbar. Note that $m_0^{\{0,0\}}, m_1^{\{0,0\}}, \dots, m_{255}^{\{0,0\}}$ are constant, irrelevant to intermediate results. Therefore, storing these values does not result in frequent updates to the RRAM, requiring only one RRAM program. During the

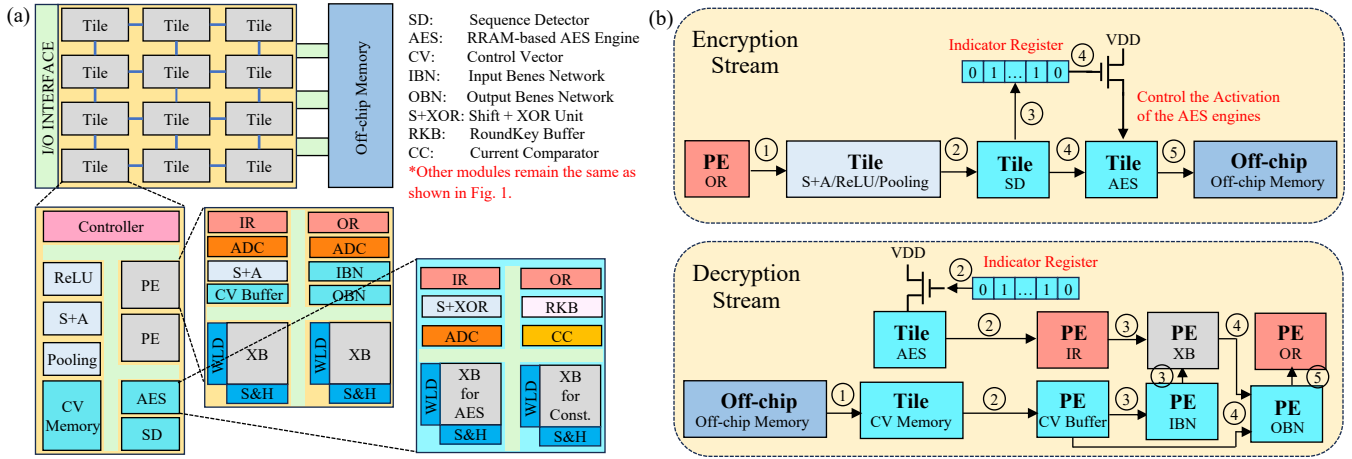


Fig. 5. (a) The Overall Architecture of *OmniGuard* and the Proposed AES Engine. (b) *OmniGuard*'s Encryption and Decryption Stream.

MixColumn operation, the controller in PE decides which rows to activate based on the value of the State and then reads out the final result. For example, if the 8-bit format of S_0 is '00000011', $R_7^{0,0}, \dots, R_2^{0,0}$ are zero. $Out^{0,0}$ equals to $R_0^{0,0} R_1^{0,0} + R_0^{0,0} R_1^{0,0}$. The controller opens corresponding rows which store m_1 and m_3 . The structure of the RRAM Crossbar automatically adds m_1 and m_3 and outputs the result. However, the addition result and the OR operation result are not equivalent. To overcome this, we use current comparators in the readout circuit to make the addition operation the same as the OR operation. Specifically, the current comparator performs a comparison against zero. If the value retrieved from the RRAM crossbar exceeds zero, this indicates the presence of at least one '1' in the corresponding column of RRAM cells. In this case, $m_1^{col_i} + m_3^{col_i}$ is equal to $m_1^{col_i} m_3^{col_i}$. This means that for each column, the result of the OR operation is identical to that of the addition operation. Thus, the current comparator outputs the addition result. In contrast, when the value from the RRAM crossbar does not exceed zero, the current comparator outputs zero. Through this mechanism, we convert the result of the addition operation into the result of the OR operation.

In *OmniGuard*, we use binary RRAM cells and assume weights are INT-8 quantization, so it needs 8 RRAM cells for each m_i . There are a total of 16 numbers in the MIX_C Matrix (from $m_i^{0,0}$ to $m_i^{3,3}$), so it needs 16×8 columns. $m_0, \dots, m_{255}$ needs 256 rows. As a result, 256×128 RRAM cells are needed to contain all m_i in the MixColumn operations and compose an AES engine. Fig. 4 (d) illustrates the details of the AES engine. Following rigorous evaluation of AES engine area requirements and available space, two AES engines are integrated into an RRAM crossbar. The setup includes two current comparators, an extra RRAM crossbar for AES operation constants, a high-speed XOR module (for RoundKeyGen/ShiftRow), and a dedicated RoundKey buffer; input/output signals are buffered by an IR and OR, respectively. Since RRAM-stored data is public and unrelated to intermediate results, the RRAM engine needs no protection, thus eliminating the requirement for bit-grained shuffling and BNs in the AES engine.

We test the performance details of the proposed AES engine. It requires only 0.05nJ of energy for a single encryption of 128 bits. When encrypting 1GB of intermediate results, its energy overhead is only 0.82J. In our tests, the AES engine's bandwidth is able to reach 71.3Gbps, which is higher than the bandwidth of the intermediate result of a single RRAM computation. Therefore, we arrange the execution pipeline of the AES engine and the accelerator like this: the RRAM accelerator is calculated in stages when it is actually executed. After the current stage, we start the AES engine to encrypt the intermediate results, and the accelerator executes the next stage in parallel. Since the AES encryption engine is faster than the calculation of a single stage, the delay of AES encryption can be completely hidden.

In this subsection, the mature AES algorithm is introduced to protect data paths and off-chip memory, followed by an analysis of its bottlenecks and characteristics. Drawing on this analysis, an RRAM-based AES engine is proposed, which allows RRAM-based accelerators to achieve effective and rapid encryption and decryption.

C. System Overview and Encryption / Decryption Stream

1) *System Overview of OmniGuard*: Fig. 5 shows the overall architecture of *OmniGuard*. Specifically, we introduced three modules in Tile: AES engine, control vector memory (CV Memory), and high-speed sequence detector (SD). In PE, we introduce a control vector buffer (CV Buffer), IBN, and OBN. The AES engine includes input registers (IR), output registers (OR), shift and XOR calculation units (S+XOR), current comparator (CC), RoundKey buffer (RKB), and the RRAM crossbar.

Furthermore, we conduct a system-level optimization for *OmniGuard*. We propose an optimization method that utilizes sparsity to reduce control vector storage overhead and encryption and decryption computation. Specifically, we can use a high-speed sequence detector to make judgments on intermediate results. If the result is 0, then the corresponding sub-array does not need to undergo the bit-grained shuffling, and the corresponding control vector does not need to be stored. Meanwhile, the zero intermediate result does not

require encryption through AES. This approach effectively reduces the computation and storage overhead of protection.

2) **Encryption / Decryption Stream:** The overall calculation process of *OmniGuard* is shown in Fig. 5, consisting of two streams: Encryption Stream and Decryption Stream. Firstly, CIM calculation is performed through a shuffled crossbar, and the result is output by PE's output register. Then, it is calculated through the Tile's special computing unit, corresponding to F1 and F2, as mentioned in Section III-A. Furthermore, the sequence detector in the tile detects the intermediate result. If the intermediate result is '0', the indicator is set to '0'. Otherwise, it is set to '1'. Then, these indicators are used to control the AES engine, where zero intermediate results do not need to be encrypted or transmitted, while non-zero intermediate results are encrypted and then transmitted to the off-chip memory through the data paths.

The encrypted intermediate result is loaded from off-chip memory, and the Tile's control vector memory transfers the corresponding control vectors to PE's control vector buffer. At the same time, the indicators control the AES engine and transmit the decrypted results to PE's input register. Then, under the function of the control vector, the IBN of PE is combined with the input data in the PE's input register to complete the calculation in PE's RRAM crossbar. Then, under the function of OBN, restore the output result to the correct CIM result. The results are transmitted to PE's output register and wait for the next stream.

This architecture and system optimization further reduce the storage and time overhead introduced by security design in *OmniGuard*, making it more efficient. In addition, this kind of design makes the control vectors' storage and encryption/decryption process in *OmniGuard* relatively random (attackers cannot know which intermediate results of the model are sparse), further enhancing its security.

V. EVALUATIONS

In this section, we implement *OmniGuard* and evaluate it through sufficient experiments. We compare the security, speed, power & area consumption, and speed performance of *OmniGuard* with SOTA studies. Furthermore, we conduct ablation studies and explore the overhead and hyper-parameters of *OmniGuard*.

A. Baselines and Setup

We choose to compare *OmniGuard* with both secure designs (including SRA [8], ChaoPIM [12], Chaotic-Weights [11] and RRAM-DES [10]) and non-secure designs (including ISAAC [5], Attar [37] and SPIKA [38]). We first implement *OmniGuard* based on Verilog. Tab. V presents the detailed parameters of *OmniGuard*. Regarding the CV buffer and the CV memory, we conservatively estimate that they respectively require 12B and 2KB. We use CACTI 6.5 [39] at 32 nm to model energy and area for all buffers and on-chip interconnects. In addition, we construct the IBN & OBN circuits using 32nm technology and evaluate their power and area by Synopsis Design Compiler. We use NeuroSim [40] to get the parameters of RRAM crossbars. We use the parameter of a convincing article [41] to implement the current comparator.

TABLE V
ARCHITECTURE PARAMETERS OF *OmniGuard*

Unit	Spec	Power(mW)	Area(mm ²)
PE			
ADC	Num:8	16	9.60×10^{-3}
DAC	Num:8*256	8	3.40×10^{-4}
S+H	Num:8*256	0.02	8.00×10^{-5}
Xbar Array	Size:8*256*256	2.7	8.00×10^{-4}
IR	Size:2KB	1.24	2.10×10^{-3}
OR	Size:256B	0.23	7.70×10^{-4}
CV Buffer	Size:12B	0.021	3.61×10^{-5}
IBN&OBN	Num:8+8	0.1632	5.89×10^{-4}
S+A	Num:4	0.2	2.40×10^{-4}
PE Total	Num:1	28.57	1.46×10^{-2}
Tile			
PE	Num:12	342.89	1.75×10^{-1}
Off-chip Mem.	Size:64KB	20.7	8.30×10^{-2}
Bus	Wires:384	7	9.00×10^{-2}
Router	Shared	42	1.51×10^{-1}
Activation	Num:2	0.52	6.00×10^{-4}
S+A	Num:1	0.05	6.00×10^{-5}
Maxpool	Num:1	0.4	2.40×10^{-4}
OR	Size:3KB	1.68	3.20×10^{-3}
CV Mem.	Size:2KB	0.647	9.67×10^{-4}
AES Engine	Num:2	20.56	6.82×10^{-3}
Seq. Detector	Num:1	0.0018	1.70×10^{-5}
Tile Total	Num:1	404.95	3.97×10^{-1}
Chip			
Tile	Num:168	68.03W	6.67×10^1
Hyper Tr.	Freq:1.6GHz	10.4W	2.28×10^1
Chip Total	Num:1	78.43W	8.96×10^1

Note that the power consumption reported in Tab. V is a static value. In actual operation, the chip power consumption is much lower than 78.43W due to our system-level optimization.

B. Results

1) **Security Evaluation:** As a secure framework, we first evaluate the security of *OmniGuard*. We choose two common DNN networks named VGG-16 [42] and ResNet-18 [43] and train them on three datasets: MNIST [44], SVHN [45], and CIFAR10 [46] and test on two levels respectively: on-chip security level and off-chip security level.

a) **On-chip Security:** First, we test the performance of *OmniGuard* in on-chip protection. We choose ChaoPIM, Secure-RRAM, and Chaotic-Weights as baselines. All three methods claim to protect RRAM crossbars in bit-level, with ChaoPIM and Chaotic-Weights using block-grained shuffling, and Secure-RRAM using XOR algorithm for array encryption. We trained both VGG-16 and ResNet-18 models on three datasets with the accuracy shown in Tab. VI, represented as 'Acc.'. We report the accuracy of inference directly after encryption, expressed as 'Acc.@Encry'. The lower the 'Acc.@Encry', the more complex the proof encryption scheme and the more difficult it is to crack. As can be seen from the results in Tab. VI, compared to the baseline, the encrypted accuracy of name is between 9.7% and 10.3%. Because the selected datasets are all ten-classification tasks, the accuracy of random sampling is ~10%. Therefore, the effect of name encryption is almost equivalent to random sampling, which proves the effectiveness of its on-chip protection.

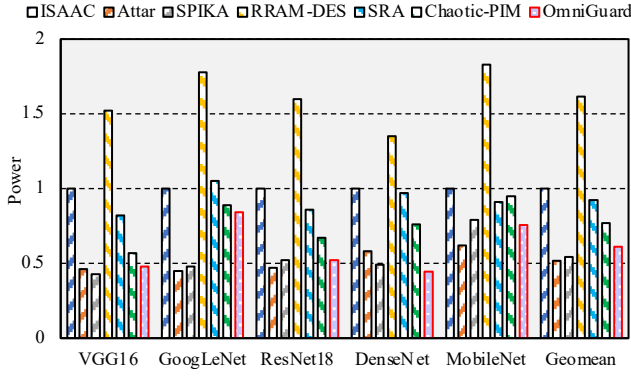


Fig. 6. Power Comparison Results among secure/non-secure baselines and the proposed *OmniGuard*.

b) Off-chip Security. Although the security of the AES algorithm has been widely proven, for the sake of integrity, we still discuss the off-chip security of *OmniGuard*. We assume that attacks targeting data paths and off-chip memory can fully capture the ciphertext encrypted by the AES engine. In this scenario, to decrypt the ciphertext and obtain the correct intermediate result, the attacker would need to make 2^{128} attempts, which are considered impossible. Therefore, we can conclude that the protection regarding off-chip security in *OmniGuard* is effective.

2) Power Evaluation. To evaluate the power and area consumption of *OmniGuard*, we build it on five DNN model scenarios, including VGG-16 [42], GoogLeNet [47], ResNet-18 [43], MobileNet [48], and DenseNet [49]. We choose three

TABLE VI
EVALUATION OF OFF-CHIP SECURITY.

Model	Dataset	Acc.	Method	Acc.@Encry.(↓)
VGG-16	MNIST	99.5%	ChaoPIM	47.6%
			Secure-RRAM	51.7%
			Chaotic-Weights	45.3%
			<i>OmniGuard</i>	9.7%
	CIFAR10	92.5%	ChaoPIM	36.1%
			Secure-RRAM	41.7%
			Chaotic-Weights	37.3%
			<i>OmniGuard</i>	9.8%
	SVHN	95.8%	ChaoPIM	38.4%
			Secure-RRAM	37.1%
			Chaotic-Weights	38.8%
			<i>OmniGuard</i>	9.7%
ResNet-18	MNIST	99.6%	ChaoPIM	49.3%
			Secure-RRAM	54.1%
			Chaotic-Weights	48.2%
			<i>OmniGuard</i>	10.1%
	CIFAR10	93.1%	ChaoPIM	40.0%
			Secure-RRAM	46.1%
			Chaotic-Weights	42.2%
			<i>OmniGuard</i>	10.3%
	SVHN	95.8%	ChaoPIM	39.5%
			Secure-RRAM	44.7%
			Chaotic-Weights	38.4%
			Bit-Shuffling	10.3%

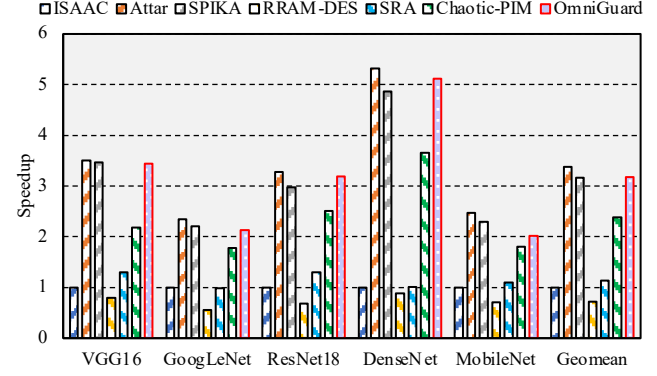


Fig. 7. Speed Comparison Results among secure/non-secure baselines and the proposed *OmniGuard*.

non-secure designs (ISAAC, Attar, SPIKA) and three secure designs (RRAM-DES, SRA, Chaotic-PIM) as baselines. We report the power consumption ratio of different operations based on the operating power consumption of ISAAC. We build a powerful architecture simulator and monitor the consumption. The results are shown in Fig. 6.

First, compared to non-secure designs (ISAAC, Attar, SPIKA), while there are some security-related overheads in the framework, *OmniGuard* still achieves similar power consumption. Second, compared to secure designs (RRAM-DES, SRA, Chaotic-PIM), *OmniGuard* achieves energy savings ranging from $1.26\times\sim 2.65\times$.

This energy saving arises from two main aspects: This energy saving stems from four key aspects. (1) The system optimization proposed in *OmniGuard* fully leverages the sparsity of DNN models, yielding significant computation savings and lower power consumption in processing elements (PEs), especially in DACs and ADCs. (2) The security designs in *OmniGuard* add minimal extra power consumption. At the on-chip security level, bit-grained shuffling is computed offline, eliminating any increase in the accelerator's computational overhead; moreover, the BNs used in PEs are extremely low-power and can be deemed negligible. (3) At the off-chip security level, *OmniGuard*'s proposed AES engine is RRAM-based, minimizing data movement for constant values in AES. (4) We also fully exploit the RRAM crossbar's characteristics in AES calculation and decomposition, using current summation to efficiently perform the most complex computations and thus reduce power consumption.

3) Speed Results: We employ a custom cycle-accurate simulator to evaluate the speedup of *OmniGuard*. We compare the speed of the aforementioned baselines and *OmniGuard* on three datasets: MNIST [44], SVHN [45], and CIFAR10 [46] and get the average results. We use the ISAAC speed as a benchmark and report the acceleration ratio of different tasks compared to ISAAC. The speed results are shown in Fig. 7.

Compared to non-secure designs, *OmniGuard* achieves a 3.18x speedup of the conventional RRAM-based accelerator (ISAAC). Moreover, *OmniGuard* remains similar speed compared to latest designs (Attar and SPIKA). Compared with secure designs, *OmniGuard* achieves $1.33\times\sim 4.38\times$ speed up, showing significantly performance improvement. This im-

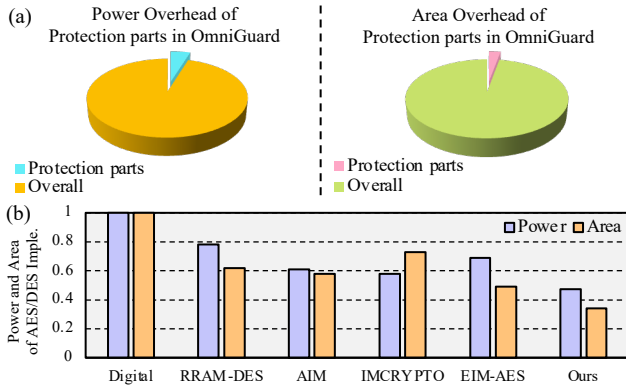


Fig. 8. (a) Power and Area Overhead of Protection Components. (b) Comparison between Our AES Engine and SOTA Studies.

provement primarily results from the system optimization we propose, which saves considerable computation and greatly reduces the computational load. Additionally, fewer intermediate results need to be transmitted off-chip, significantly reducing the time overhead of the proposed RRAM-based AES engine, and further accelerating the overall speed of the accelerator.

C. Ablation Study

1) **Overhead of Protection Components:** To evaluate the efficiency of the security designs in *OmniGuard*, we conduct two experiments. First, we separate the protection components (including BNs, CV Memory, CV Buffer, and the AES engine) within the overall framework and compare their power consumption and area proportions in the entire accelerator, as shown in Fig 8 (a). Second, we select SOTA AES/DES engine (including a Digital-implementation [50] and several engines based on non-volatile-memory [10], [34]–[36]) to compare with the proposed AES engine in *OmniGuard*, and the results are presented in Fig. 8 (b).

From Fig. 8 (a), it is obvious that the overhead of protection parts in *OmniGuard* is relative small. *OmniGuard* uses only 5% of the power and 3% of the area to achieve protection at both the on-chip and off-chip levels. This overhead can be ignored in the overall architecture, which proves the high efficiency of our security-related designs. Moreover, our AES engine shows advantages in both power and area aspects compared to the SOTA studies. Compared with SOTA studies, our AES engine achieves a $1.23\times\sim 2.13\times$ power saving and $1.44\times\sim 2.94\times$ area saving. This proves that our proposed AES engine has better efficiency and can provide security protection with minimal overhead.

2) **Effectiveness of System Optimization:** We tested the power consumption and speed of *OmniGuard* with (w/) and without (w/o) system optimization to verify the effectiveness of system optimization. Tab. VII shows the results. We can see that on all five models, after adding system optimization, the power consumption is reduced $1.54\times\sim 2.33\times$, and the inference speed is increased $1.81\times\sim 3.00\times$. This proves that the system optimization in name can effectively reduce the computational cost of encryption and decryption, resulting in practical power savings and speed improvement.

TABLE VII
ABLATION STUDY OF THE SYSTEM OPTIMIZATION (SO).

Model	OmniGuard w/o SO		OmniGuard w/ SO	
	Power	Speed	Power	Speed
VGG16	76.93W	0.38ms	36.75W	0.21ms
GoogLeNet	79.31W	0.35ms	41.86W	0.19ms
ResNet-18	72.87W	0.05ms	37.86W	0.02ms
DenseNet	79.96W	0.20ms	34.38W	0.09ms
MobileNet	73.46W	0.003ms	47.71W	0.001ms

TABLE VIII
ROBUSTNESS EVALUATION AGAINST RRAM DEVICE VARIATION.

Model	5% RRAM Noise		10% RRAM Noise	
	AES Err.	E2E Acc.	AES Err.	E2E Acc.
VGG16	0.12%	↓ 0.27%	1.23%	↓ 3.29%
GoogLeNet	0.15%	↓ 0.42%	0.97%	↓ 4.03%
ResNet-18	0.10%	↑ 0.14%	1.46%	↓ 3.75%
DenseNet	0.13%	↓ 0.79%	0.87%	↓ 2.98%
MobileNet	0.11%	↑ 0.53%	1.53%	↓ 4.02%

D. Discussion

1) **Hyperparameters in OmniGuard:** In *OmniGuard*, there are two hyperparameters: the number of IBN/OBN in each PE and the number of AES engines in each tile. To identify the optimal scheme, we adjusted these two hyperparameters and tracked corresponding changes in the accelerator's calculation speed, with the results presented in Fig. 9. We selected the configuration of one AES engine per tile and two IBNs/OBNs per PE as the initial setup. Initially, the performance bottleneck was associated with the number of BNs under the configuration 1A-2I-2O (denoting one AES engine per tile and two IBNs/OBNs per PE), and the calculation speed increased as the number of BNs rose. However, when the number of IBNs/OBNs reached four, the bottleneck shifted to the number of AES engines (configuration 1A-4I-4O). In this scenario, further increasing the number of BNs (as in the 1A-6I-6O configuration) no longer improved the speed. Specifically, without increasing the number of AES engines, adding more BNs failed to enhance the accelerator's calculation speed.

Yet, when the number of AES engines increases to two, the number of BNs once again becomes the bottleneck. Expanding

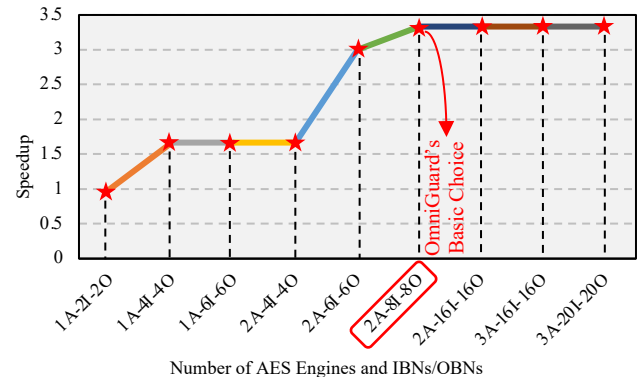


Fig. 9. The Performance of *OmniGuard* under Different Hyperparameters (Number of IBN/OBN and Number of AES engine).

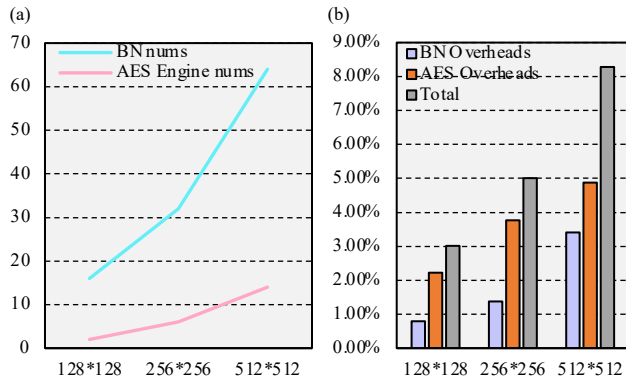


Fig. 10. (a) Numbers of BNs and AES Engines Varying with Crossbar Size (b) Overheads of BNs and AES Engines Varying with Crossbar Size

the number of BNs can enhance speed performance, as seen in the configurations 2A-4I-4O, 2A-6I-6O, and 2A-8I-8O. However, a further increase in the number of both AES engines and BNs will overload the off-chip router, resulting in unsustained speed growth—this is evident in configurations such as 2A-16I-16O, 3A-16I-16O, and 3A-20I-20O. Additionally, more AES engines and BNs are associated with higher power and area consumption. Consequently, *OmniGuard* selects 2A-8I-8O as the standard solution, with the other configurations serving as alternatives

2) **Robustness against RRAM Device Variation:** In real-world scenarios, RRAM devices often experience resistance variations due to environmental changes. Therefore, we tested the robustness of name to changes in RRAM devices. Specifically, we randomly added 5% and 10% noise to each device in the AES engine, and then observed the correctness of the encryption results. Tab. VIII shows the results. With 5% RRAM noise, the error rate of AES engine is maintained at 0.10%~0.15%. This is because the name is not based on accurate calculation to achieve AES, but through the current comparator at its output to obtain the final result, so it has a certain filtering ability. Therefore, there are very few AES errors, resulting in very little change in end-to-end accuracy. With 10% RRAM noise, the error rate of AES engine enlarge to 0.87%~1.53%, leading to accuracy degradation. Overall, *OmniGuard* demonstrates a certain degree of robustness against RRAM device variations.

3) **Scalability Testing:** We evaluate *OmniGuard*'s security overhead across three crossbar sizes (128×128, 256×256, 512×512). With 8-bit BNs whose count scales with crossbar size, both BN and control vector storage overhead grow. AES engine count also increases with crossbar size. Fig. 10 (a) shows BN count growing faster than AES count as crossbar size increases, since both input and output sides require more BNs. Fig. 10 (b) shows BN and AES power overhead increasing with crossbar size but remaining low (below 10%).

VI. CONCLUSION

In this paper, we propose a multi-level protection framework for the RRAM-based accelerators, which is called *OmniGuard*. We propose a unified graph-based representation to segment the security level. For on-chip security, *OmniGuard* utilizes a bit-grained shuffling to achieve protection and adopts Benes

Networks to restore the CIM capability. For off-chip security, *OmniGuard* proposes an novel RRAM-based AES engine. Experiments show that *OmniGuard* achieves $1.33\times\sim4.38\times$ speedup and $1.26\times\sim2.65\times$ power savings.

VII. ACKNOWLEDGMENTS

This work is supported by National Key R& D Program of China: 2024YFB4505800, National Natural Science Foundation of China (62341407, 62322401, U24B20166, 62406008), in part by the "111" Project (B18001).

REFERENCES

- [1] Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, "Object detection in 20 years: A survey," *Proceedings of the IEEE*, vol. 111, no. 3, pp. 257–276, 2023.
- [2] C. Zheng, W. Wu, C. Chen, T. Yang, S. Zhu, J. Shen, N. Kehtarnavaz, and M. Shah, "Deep learning-based human pose estimation: A survey," *ACM Computing Surveys*, vol. 56, no. 1, pp. 1–37, 2023.
- [3] R. Prabhavalkar, T. Hori, T. N. Sainath, R. Schlüter, and S. Watanabe, "End-to-end speech recognition: A survey," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2023.
- [4] T. Mohaidat and K. Khalil, "A survey on neural network hardware accelerators," *IEEE Transactions on Artificial Intelligence*, 2024.
- [5] A. Shafiee, A. Nag, N. Muralimanohar, R. Balasubramanian, J. P. Strachan, M. Hu, R. S. Williams, and V. Srikumar, "Isaac: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars," in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 14–26.
- [6] P. Chi, S. Li, C. Xu, T. Zhang, J. Zhao, Y. Liu, Y. Wang, and Y. Xie, "Prime: A novel processing-in-memory architecture for neural network computation in rram-based main memory," in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 27–39.
- [7] W. Li, S. Huang, X. Sun, H. Jiang, and S. Yu, "Secure-rram: A 40nm 16kb compute-in-memory macro with reconfigurability, sparsity control, and embedded security," in *2021 IEEE Custom Integrated Circuits Conference (CICC)*, 2021, pp. 1–2.
- [8] L. Zhao, Y. Zhang, and J. Yang, "Sra: a secure rram-based dnn accelerator," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, ser. DAC '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 355–360. [Online]. Available: <https://doi.org/10.1145/3489517.3530440>
- [9] P. Sheridan, K.-H. Kim, S. Gaba, T. Chang, L. Chen, and W. Lu, "Device and spice modeling of rram devices," *Nanoscale*, vol. 3, no. 9, pp. 3833–3840, 2011.
- [10] H. Bi, P. Wang, J. Ye, L. Zhao, Y. Zhang, and B. Chen, "A rram-based logic in memory des implementation against power attacks," *Available at SSRN 5097243*.
- [11] N. Lin, X. Chen, H. Lu, and X. Li, "Chaotic weights: A novel approach to protect intellectual property of deep neural networks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 40, no. 7, pp. 1327–1339, 2021.
- [12] N. Lin, X. Chen, C. Xia, J. Ye, and X. Li, "Chaopim: A pim-based protection framework for dnn accelerators using chaotic encryption," in *2021 IEEE 30th Asian Test Symposium (ATS)*. IEEE, 2021, pp. 1–6.
- [13] S. Yu, W. Shim, X. Peng, and Y. Luo, "Rram for compute-in-memory: From inference to training," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 68, no. 7, pp. 2753–2765, 2021.
- [14] Z. Li, Z. Wang, L. Xu, Q. Dong, B. Liu, C.-I. Su, W.-T. Chu, G. Tsou, Y.-D. Chih, T.-Y. J. Chang, D. Sylvester, H.-S. Kim, and D. Blaauw, "Rram-dnn: An rram and model-compression empowered all-weights-on-chip dnn accelerator," *IEEE Journal of Solid-State Circuits*, vol. 56, no. 4, pp. 1105–1115, 2021.
- [15] P.-Y. Chen and S. Yu, "Compact modeling of rram devices and its applications in 1t1r and 1s1r array design," *IEEE Transactions on Electron Devices*, vol. 62, no. 12, pp. 4022–4028, 2015.
- [16] Y. Deng, P. Huang, B. Chen, X. Yang, B. Gao, J. Wang, L. Zeng, G. Du, J. Kang, and X. Liu, "Rram crossbar array with cell selection device: A device and circuit interaction study," *IEEE transactions on Electron Devices*, vol. 60, no. 2, pp. 719–726, 2012.
- [17] Z. Shen, C. Zhao, Y. Qi, W. Xu, Y. Liu, I. Z. Mitrovic, L. Yang, and C. Zhao, "Advances of rram devices: Resistive switching mechanisms, materials and bionic synaptic application," *Nanomaterials*, vol. 10, no. 8, p. 1437, 2020.

- [18] M. Gao, M. S. Rahman, N. Varshney, M. Tehranipoor, and D. Forte, "Iprobe: internal shielding approach for protecting against front-side and back-side probing attacks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 12, pp. 4541–4554, 2023.
- [19] A. Almazayad, L. Halman, and A. Alsaeed, "Probe attack detection using an improved intrusion detection system," *Computers, Materials & Continua*, vol. 74, no. 3, 2023.
- [20] P. Zuo, Y. Hua, L. Liang, X. Xie, X. Hu, and Y. Xie, "Sealing neural network models in encrypted deep learning accelerators," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 1255–1260.
- [21] Y. Jiang, H. Zhu, D. Sullivan, X. Guo, X. Zhang, and Y. Jin, "Quantifying rowhammer vulnerability for dram security," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 73–78.
- [22] R. Zhou, S. Ahmed, A. Siraj Rakin, and S. Angizi, "Dnn-defender: An in-dram deep neural network defense mechanism for adversarial weight attack," *arXiv e-prints*, pp. arXiv–2305, 2023.
- [23] R. Gayatri, Y. Gayatri, C. Mitra, S. Mekala, and M. Priyatharishini, "System level hardware trojan detection using side-channel power analysis and machine learning," in *2020 5th International Conference on Communication and Electronics Systems (ICCSES)*, 2020, pp. 650–654.
- [24] G. Khedkar and D. Kudithipudi, "Rram motifs for mitigating differential power analysis attacks (dpa)," in *2012 IEEE Computer Society Annual Symposium on VLSI*. IEEE, 2012, pp. 88–93.
- [25] N. Huynh, H. Cherian, and E. C. Ahn, "Hardware security of emerging non-volatile memory devices under imaging attacks," in *2021 International Conference on Applied Electronics (AE)*. IEEE, 2021, pp. 1–4.
- [26] S. Barve, S. Shukla, S. M. P. Dinakarrao, and R. Jha, "Adversarial attack mitigation approaches using rram-neuromorphic architectures," in *Proceedings of the 2021 Great Lakes Symposium on VLSI*, 2021, pp. 201–206.
- [27] R. Liu, H. Wu, Y. Pang, H. Qian, and S. Yu, "A highly reliable and tamper-resistant rram puf: Design and experimental validation," in *2016 IEEE international symposium on hardware oriented security and trust (HOST)*. IEEE, 2016, pp. 13–18.
- [28] M. Jahanshahi and F. Bistouni, "Improving the reliability of the benes network for use in large-scale systems," *Microelectronics Reliability*, vol. 55, no. 3–4, pp. 679–695, 2015.
- [29] P. D. Manuel, M. I. Abd-El-Barr, I. Rajasingh, and B. Rajan, "An efficient representation of benes networks and its applications," *Journal of Discrete Algorithms*, vol. 6, no. 1, pp. 11–19, 2008.
- [30] A. M. Abdullah *et al.*, "Advanced encryption standard (aes) algorithm to encrypt and decrypt data," *Cryptography and Network Security*, vol. 16, no. 1, p. 11, 2017.
- [31] P. Kumar and S. B. Rana, "Development of modified aes algorithm for data security," *Optik*, vol. 127, no. 4, pp. 2341–2345, 2016.
- [32] K. Iwai, N. Nishikawa, and T. Kurokawa, "Acceleration of aes encryption on cuda gpu," *International Journal of Networking and Computing*, vol. 2, no. 1, pp. 131–145, 2012.
- [33] H. T. Assafii, I. A. Hashim, and A. A. Naser, "Advanced encryption standard (aes) acceleration and analysis using graphical processing unit (gpu)," *Applied Nanoscience*, pp. 1–6, 2021.
- [34] D. Reis, H. Geng, M. Niemier, and X. S. Hu, "Imcrypto: An in-memory computing fabric for aes encryption and decryption," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 30, no. 5, pp. 553–565, 2022.
- [35] M. Ma, Y. Zhu, Z. Zhu, R. Yuan, J. Liu, L. Xu, Y. Yang, and Y. Wang, "Efficient in-memory aes encryption implementation using a general memristive logic: Surmounting the data movement bottleneck," *IEEE Nanotechnology Magazine*, vol. 16, no. 2, pp. 24–C3, 2022.
- [36] —, "Efficient in-memory aes encryption implementation using a general memristive logic: Surmounting the data movement bottleneck," *IEEE Nanotechnology Magazine*, vol. 16, no. 2, pp. 24–C3, 2022.
- [37] B. Li, Y. Qi, Y. Wang, and Y. Han, "Attar: Rram-based in-memory attention accelerator with software-hardware co-optimization," *Science China Information Sciences*, vol. 68, no. 3, p. 132401, 2025.
- [38] K. Humood, P. Foster, S. Wang, A. Serb, and T. Prodromakis, "Spika: 200-tops/w rram-based neural network accelerator chip," in *2025 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2025, pp. 1–5.
- [39] N. Muralimanohar, R. Balasubramonian, and N. Jouppi, "Optimizing nuca organizations and wiring alternatives for large caches with cacti 6.0," in *40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2007)*, 2007, pp. 3–14.
- [40] P.-Y. Chen, X. Peng, and S. Yu, "Neurosim: A circuit-level macro model for benchmarking neuro-inspired architectures in online learning," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 12, pp. 3067–3080, 2018.
- [41] S. Sarkar and S. Banerjee, "500 mhz differential latched current comparator for calibration of current steering dac," in *Proceedings of the 2014 IEEE Students' Technology Symposium*, Feb 2014, pp. 309–312.
- [42] S. Liu and W. Deng, "Very deep convolutional neural network based image classification using small training sample size," in *2015 3rd IAPR Asian Conference on Pattern Recognition (ACPR)*, 2015, pp. 730–734.
- [43] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [44] L. Deng, "The mnist database of handwritten digit images for machine learning research [best of the web]," *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 141–142, 2012.
- [45] Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, and A. Ng, "Reading digits in natural images with unsupervised feature learning," 2011. [Online]. Available: <https://api.semanticscholar.org/CorpusID:16852518>
- [46] A. Krizhevsky, "Learning multiple layers of features from tiny images," 2009. [Online]. Available: <https://api.semanticscholar.org/CorpusID:18268744>
- [47] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 1–9.
- [48] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," 2017.
- [49] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2261–2269.
- [50] A. Singh, M. Kar, S. Mathew, A. Rajan, V. De, and S. Mukhopadhyay, "25.3 a 128b aes engine with higher resistance to power and electromagnetic side-channel attacks enabled by a security-aware integrated all-digital low-dropout regulator," in *2019 IEEE International Solid-State Circuits Conference - (ISSCC)*, 2019, pp. 404–406.



Zihao Zheng received the B.Eng. degree from Beijing Jiaotong University, Beijing, China, in 2022. And he his M.S. degree from Peking University, Beijing, China, in 2025. He is currently a Ph.D student at the School of Computer, Peking University, Beijing, China.

His research interests include In-memory computing, computer architecture, edge computing , electronic design automation and artificial intelligence.



Ling Liang received the B.Eng. degree from Beijing University of Posts and Telecommunications, Beijing, China, in 2015 and M.S. degree from University of Southern California, CA, USA in 2017. He received his Ph.D. degree from the University of California Santa Barbara, CA, USA, in 2022.

He is currently an research assistant professor at the School of Integrated Circuits, Peking University, Beijing, China.



Yunfan Yang received the Ph.D. degree in electronic engineering from Peking University, Beijing, China, in 2025. After that, he joined the School of Integrated Circuits, Peking University, Beijing, China, as a Post-Doctoral Assistant Researcher.

His research interests include emerging memory technology, in-memory computing circuits, VLSI design, and embodied intelligence.



Jinlong Lin received his M.S. degree and the Ph.D. degree from Peking University, Beijing, China.

He is currently a professor at the School of Integrated Circuits, Peking University, Beijing, China. His research interests include embedded systems, computing vision, and digital image analysis.



Meng Li received the B.Eng. degree from Peking University, Beijing, China, in 2013. He received his M.S. degree from The University of Texas at Austin, Austin, TX, USA, in 2017. He received his M.S. degree from The University of Texas at Austin, Austin, TX, USA, in 2018.

He is currently an assistant professor at the School of Integrated Circuits, Peking University, Beijing, China.



Zongwei Wang received the B.Eng. degree from Xidian University, Xi'an, China. He received his M.S. and Ph.D. degrees from Peking University, Beijing, China.

He is currently an assistant professor at the School of Integrated Circuits, Peking University, Beijing, China.



Yimao Cai received B.Eng. degree in 2000 and M.S. degree in 2013 from Beijing Normal University, Beijing, China. He received his Ph.D. degree from Peking University, Beijing, China, in 2006. He is currently a Professor at the School of Integrated Circuits, Peking University, Beijing, China. His research interests are key technologies of advanced memory, Neuromorphic Devices, and intelligent chips.